Shri Vile Parle Kelavani Mandal's
DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

**Department of Artificial Intelligence (AI) and Data Science**

# Learnova: Technical Architecture and System Documentation

How the system works, the technology stack and the rules it applies

Swayam Devendra Panchal (60018240013)

Kawaljeet Singh Bharaj (60018240016)

Jeevesh Bharat Bodhani (60018240096)

Rishit Pankajkumar Lahoti (6001840101)

Department of Artificial Intelligence and Data Science (Head of Department: Dr. Pratik Kanani), Dwarkadas J. Sanghvi College of Engineering, Mumbai — October 2026

# Contents

**1 Introduction** **2**
1.1 Purpose of this document . . . 2
1.2 What the system does . . . 3
1.3 Design principles . . . 3

**2 System overview** **4**
2.1 High-level architecture . . . 4
2.2 Life of a request . . . 5
2.3 Layered view of the engine . . . 6

**3 Technology stack** **7**
3.1 Backend and engine . . . 7
3.2 AI providers (all optional, free tiers only) . . . 8
3.3 Frontend . . . 8
3.4 Generated web deck . . . 9
3.5 System tools and infrastructure . . . 9

**4 Repository layout** **9**

**5 The processing pipeline** **10**
5.1 Stages . . . 10
5.2 Stage 1: reading the source . . . 12
5.3 Stage 2: planning the slides . . . 12

**6 The content decision engine** **13**
6.1 Role . . . 13
6.2 Visual families . . . 13
6.3 Detector order . . . 14
6.4 Rules for flowcharts, algorithms, code and equations . . . 15

**7 Enrichment** **15**
7.1 Key terms and glossary . . . 15
7.2 Checkpoint quizzes . . . 15
7.3 Discussion prompts . . . 16
7.4 Step-by-step explainers . . . 16
7.5 Interactive labs . . . 16
7.6 Illustrations and topic diagrams . . . 16

**8 Rendering** **16**
8.1 PowerPoint builder . . . 16
8.2 Animations and transitions . . . 17
8.3 Build isolation . . . 17

8.4 Interactive web deck . . . . . 17

**9 The AI layer** **18**
9.1 Offline-first escalation . . . . . 18
9.2 Provider router and key pools . . . . . 18
9.3 Checks on AI output . . . . . 19

**10 Quality measurement** **19**
10.1 Pedagogical Slide Fitness (PSF) . . . . . 19
10.2 Layout audit . . . . . 19
10.3 Fact retention and before/after . . . . . 20
10.4 Automated tests . . . . . 20

**11 Backend service** **20**
11.1 API endpoints . . . . . 20
11.2 Data and storage . . . . . 22
11.3 Security . . . . . 22

**12 Frontend** **23**
12.1 Structure . . . . . 23
12.2 Responsive design and PWA . . . . . 23

**13 Rules applied to every deck** **23**
13.1 Content rules . . . . . 24
13.2 Visual rules . . . . . 24
13.3 Layout rules . . . . . 24
13.4 Interactivity rules . . . . . 24
13.5 AI rules . . . . . 25

**14 Configuration** **25**

**15 Running and deployment** **26**
15.1 Local development . . . . . 26
15.2 Docker . . . . . 26
15.3 Free hosting . . . . . 26

**16 Known limitations and future work** **27**

# 1. Introduction

## 1.1 Purpose of this document

This document explains how Learnova works from the inside. It is written for a developer, a reviewer or an examiner who wants to know what happens between the moment a teacher uploads an old lecture

file and the moment a finished, interactive deck comes back. It covers the system architecture, every stage of the processing pipeline, the technology stack, the data and storage model, the API, the frontend, the rules the system applies to every slide, configuration, deployment and testing.

The project report describes *why* Learnova was built and what it achieved. The two research papers describe the methods in formal terms. This document sits between them: it maps each idea to the module that implements it, so that a reader can open the repository and find the code behind any behaviour.

## 1.2 What the system does

Learnova takes a teaching source and turns it into a presentation that teaches. The inputs it accepts are:

- PowerPoint files (`.pptx`), including decks full of build-up slides, tables, charts and equations;
- lecture slides exported to PDF, where bullets are glyphs in a symbol font and tables are often pictures;
- report-style PDFs, Word documents (`.docx`), Markdown and plain text;
- a one-line topic typed into the composer.

From one input it produces three outputs:

1. an **editable PowerPoint** with native shapes, tables, charts, real equations, speaker notes, slide transitions and click-by-click builds;
2. an **interactive web deck**, a single standalone HTML file with reveals, flip cards, quizzes, matching activities, step-by-step explainers and interactive labs;
3. a **study pack**: flashcards (CSV and Anki), a quiz bank (CSV and JSON), a one-page cheat sheet and a narration script.

## 1.3 Design principles

Five principles shape every part of the code.

**Keep every important point.** The system restructures and redraws content; it does not summarise it away. Text a visual cannot carry is placed in an "In plain words" panel or in the speaker notes, and a fact-retention check counts the figures, dates, formulae and terms kept on each slide.

**Offline first.** Every decision has a rule-based path that runs with no network and no API key. A language model is called only when a specific check says the offline result is weak, and only free API keys are used.

**Every number is computed.** Worked examples, quiz answers and explainer steps come from exact calculation (SymPy or exact fractions), never from a model's guess.

**Check AI output before use.** Generated SVG is parsed and sanitised, generated JavaScript is syntax-

checked with Node, generated text is grounded against the source. Anything that fails a check falls back to the offline result.

**One engine, many front ends.** Everything under `src/learnova/` is independent of any user interface. The FastAPI backend, the command-line scripts and the test suite all call the same pipeline.

# 2. System overview

## 2.1 High-level architecture

Learnova is a three-tier web application with a document-processing engine at its core.

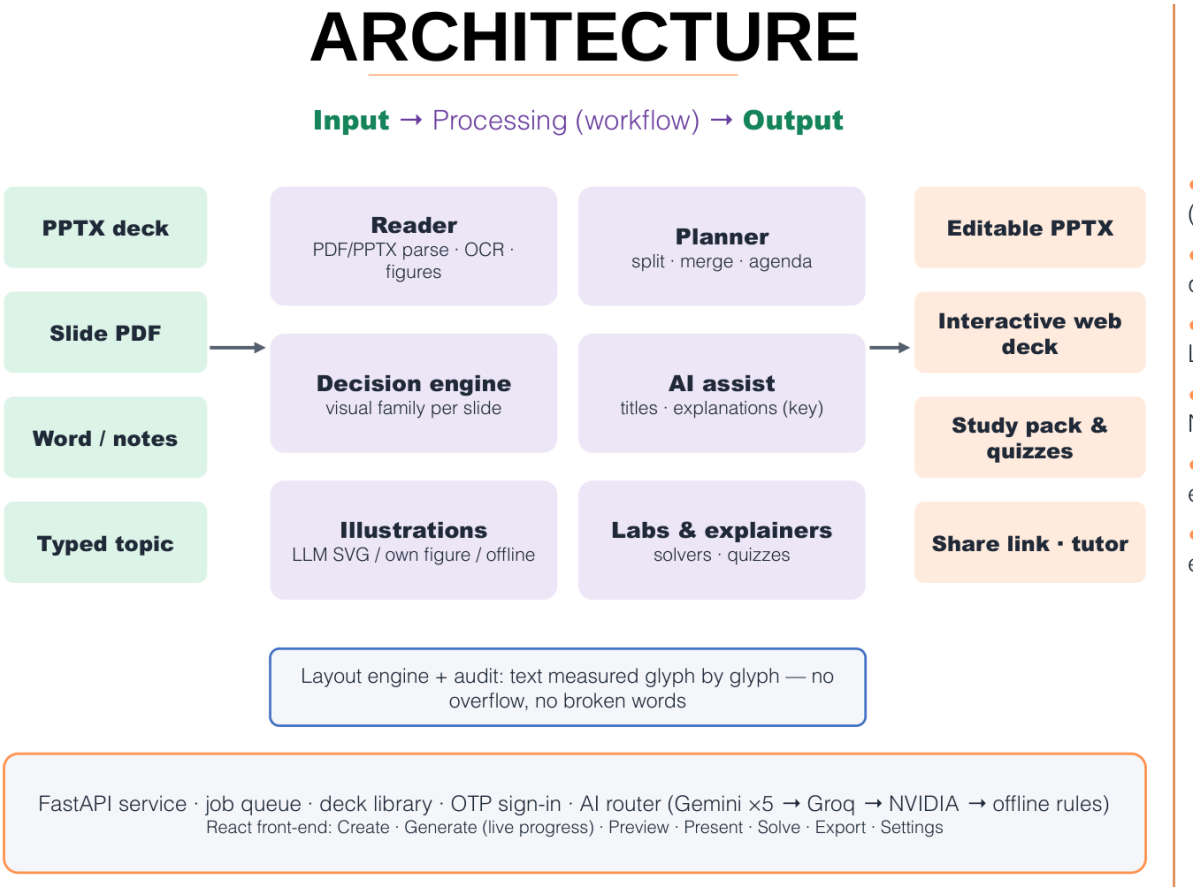



Figure 1: Learnova architecture: the React client talks to the FastAPI service, which runs the pipeline in background jobs and stores results per user.

| Tier | Component | Responsibility |
|---|---|---|
| Presentation | React 18 single-page app (Vite, Tailwind CSS, Radix UI) | Upload, composer, live progress, preview, presenter console, templates, settings; installable as a PWA |
| Application | FastAPI service (`apps/api/main.py`) | Authentication, job queue, deck library, downloads, share links, explainers, labs, AI-key management |
| Engine | Python package `learnova` (`src/learnova/`) | Parsing, planning, visual decisions, enrichment, rendering, scoring |
| Data | SQLite database and per-user folders under `.data/` | Accounts, sessions, job records, saved decks with versions, share tokens |
| External (optional) | Groq, Gemini, NVIDIA NIM free tiers | Used only on escalation, through one router with key rotation |
| System tools | LibreOffice, Tesseract OCR, Node.js | PPTX to PDF thumbnails, OCR of picture slides, checking generated lab code |

## 2.2 Life of a request

The following sequence shows what happens when a teacher uploads a file.

1. The browser sends the file to `POST /api/jobs`. The API validates the type and size, stores the upload and answers **202 Accepted** with a job id at once; it never makes the browser wait for a build.
2. A background worker converts the file into the **Markdown intermediate representation** (IR). The job moves to the state *awaiting review* and the browser, which polls `GET /api/jobs/{id}`, shows the Markdown in an editor.
3. The teacher may edit headings or text, then clicks **Generate**, which calls `POST /api/jobs/{id}/generate`.
4. The orchestrator runs the remaining pipeline stages, reporting each stage and a progress fraction to the job record.
5. PowerPoint and web-deck builds run in a **separate Python interpreter** (`rendering/subprocess_builder.p`) so a crash in a C extension cannot take the API down.

6. The finished deck, its Markdown, its JSON payload and both files are saved under `.data/users/<user>/<deck>/` and appear in **My Decks**. The browser opens the preview.

Job state is mirrored into SQLite (`pipeline/job_db.py`), so a server restart does not lose an upload: a job awaiting review comes back, and a job that was mid-build is reported as interrupted with its Markdown intact so one click rebuilds it.

## 2.3 Layered view of the engine

| Layer | Packages | Role |
|---|---|---|
| Input | `parsers/` | Read PPTX, PDF (report and slide), DOCX, Markdown, text into one IR |
| Planning | `pipeline/planner.py`, `slide_chunks.py`, `density.py` | One idea per slide; split, merge, collapse build-ups |
| Understanding | `visuals/engine.py`, `intelligence/`, `visual_specs/` | Decide the visual family for each slide and build its data |
| Enrichment | `pipeline/enrich.py`, `ai/quiz_offline.py`, `explain/`, `labs/`, `ai/topic_art.py` | Key terms, quizzes, discussion prompts, explainers, labs, illustrations |
| Direction | `rendering/deck_director.py`, `pipeline/polish.py` | Whole-deck decisions: titles, transitions, animation order, notes |
| Rendering | `rendering/ppt_builder.py`, `pptx_visuals.py`, `family_blocks.py`, `web_deck_builder.py` | Draw PPTX shapes and the HTML deck |
| Quality | `scoring/psf.py`, `evaluation/layout_audit.py`, `analysis/` | PSF score, layout audit, before/after comparison, accessibility |
| AI | `ai/escalation.py`, `providers/` | Decide when to call a model; route, rotate keys, cache |

# 3. Technology stack

## 3.1 Backend and engine

| Technology | Version | Used for |
|---|---|---|
| Python | 3.12 | All backend and engine code |
| FastAPI | 0.115+ | REST API, request validation, OpenAPI docs at `/docs` |
| Uvicorn | 0.30+ | ASGI server |
| python-multipart | 0.0.9+ | File uploads |
| SQLite (standard library) | — | Accounts, sessions, job records |
| python-pptx | 1.0.2 | Reading source decks and writing the output PowerPoint |
| PyMuPDF (fitz) | 1.25.3 | PDF text with positions and fonts, tables, images, page rendering |
| firecrawl-anydoc | 0.1.9+ | Fast document-to-Markdown conversion (pure Rust, no model) |
| python-docx | — | Reading Word documents natively |
| SymPy | — | Exact computation in step-by-step explainers |
| resvg-py | 0.5.0 | Rendering SVG diagrams (gradients, markers) to PNG for PowerPoint |
| Pillow | 10+ | Image handling, thumbnails, colour analysis |
| PyYAML | 6+ | Visual catalogue and master-prompt configuration |
| PyJWT | 2.8+ | Token verification (kept for the optional Clerk mode) |
| python-dotenv | 1.0.1 | Loading `.env` settings |

| | | |
|---|---|---|
| requests | 2.31+ | NVIDIA NIM REST calls and other HTTP |

## 3.2 AI providers (all optional, free tiers only)

| Provider | SDK | Default model | Role |
|---|---|---|---|
| Groq | `groq` 0.11+ | `openai/gpt-oss-20b` (fast `llama-3.1-8b-instant` for short JSON) | Rewriting, titles, quizzes, explainers; pool of keys |
| Google Gemini | `google-genai` 1.0+ | `gemini-3.5-flash-lite` | Vision (picture slides), topic diagrams; pool of keys paced 4.2 s apart |
| NVIDIA NIM | plain `requests` (OpenAI-compatible endpoint) | `nvidia/nemotron-3.5-lightning-30b-a3b` | Failover for any task |

There is no OpenAI SDK dependency. All model calls go through `providers/router.py`, which tries providers in a task-specific order and falls through on rate limits, time-outs and server errors.

## 3.3 Frontend

| Technology | Used for |
|---|---|
| React 18 | Component UI |
| Vite 5 | Development server (port 5190) with `/api` proxy, production build |
| React Router 7 | Client-side routing (`/app`, `/app/library`, `/app/gallery`, deck pages) |
| Tailwind CSS 4 | Styling with design tokens |
| Radix UI primitives | Accessible dialogs, menus, tabs, tooltips, selects |
| Framer Motion, GSAP, Lenis | Page and landing animations, smooth scrolling |
| Recharts | Analytics charts |
| Mermaid | Diagrams in documentation views |

| Technology | Used for |
|---|---|
| Lucide icons | Interface icons (also the offline illustration library in the engine) |
| Sonner | Toast notifications |
| Service worker + manifest | Installable Progressive Web App, offline app shell |

### 3.4 Generated web deck

The web deck is one self-contained HTML file built on **Reveal.js** with **MathML** for equations, inline SVG for diagrams and flowcharts, and plain JavaScript for quizzes, flip cards, matching, labs and explainers. It needs no server to open.

### 3.5 System tools and infrastructure

| Tool | Used for |
|---|---|
| LibreOffice (headless) | Converting PPTX to PDF for thumbnails and previews |
| Tesseract OCR | Reading text from picture-only slides before any vision model is tried |
| Node.js 20 | Building the frontend; syntax-checking AI-written lab JavaScript |
| Docker | One container with API, built web app, LibreOffice and Tesseract |
| Hugging Face Spaces / Render / Vercel | Free hosting options (`Dockerfile`, `render.yaml`, `frontend/vercel.json`) |
| pytest | 774 automated tests (6 skipped where optional tools are missing) |
| Git and GitHub | Version control (`SDP42/LearnovaV2`) |

## 4. Repository layout

```
LearnovaV2/
  apps/api/main.py     FastAPI app: auth, jobs, decks, shares, labs
```

```
src/learnova/
  config.py          paths, limits, runtime env, .env loading
  textutils.py       text cleaning, trimming, readability check
  parsers/           PPTX, PDF, slide PDF, DOCX, text -> Markdown IR
  pipeline/          orchestrator, planner, chunks, density,
                     enrich, polish, jobs, job_db
  visuals/           decision engine, math formatter, icons
  rendering/         PPTX + web deck builders, layout fitting,
                     animation, deck director
  ai/                escalation, quizzes, topic art, OCR, tutor
  providers/         Groq, Gemini, NVIDIA providers + router
  explain/           problem detection, exact solvers, stepper
  labs/              built-in labs, on-the-spot lab generation
  analysis/          before/after, accessibility, colour DNA
  scoring/           PSF metric, engagement scorer
  evaluation/        layout audit, visual-selection accuracy
  storage/           deck library, versions, share links
  auth/              phone + OTP sign-in (Clerk mode optional)
  assistant/         chat and voice assistant
  gallery/           ready-made topic catalogue
  export/            study pack
frontend/            React + Vite app, landing page, PWA files
data/samples/        sample lecture files and finished decks
tests/               pytest suite and gold fixtures
scripts/             previews, end-to-end check, research
docs/                guides, rules, report, papers, this file
```

# 5. The processing pipeline

## 5.1 Stages

The orchestrator (`pipeline/orchestrator.py`) runs sixteen named stages. The first runs before the review step; the rest run after the teacher clicks Generate. Each stage reports its name and progress to the job.

| # | Stage | What it does |
|---|---|---|
| 1 | `convert` | Reads the source into the Markdown IR, extracts images and anchors them to sections |

| # | Stage | What it does |
|---|---|---|
| 2 | `chunk` | Splits the IR on headings into slide-sized chunks; merges build-up slides |
| 3 | `vision_ocr` | Reads text from picture slides (Tesseract first, a vision model only if unreadable) |
| 4 | `index` | Builds the keyword index used by the tutor and the grounding checks |
| 5 | `layout` | Assigns a coarse layout class per chunk |
| 6 | `visual_plan` | Runs the content decision engine: one visual family per slide, with a reason |
| 7 | `visual_data` | Builds the data each visual needs (steps, rows, nodes, tiles, equations) |
| 8 | `expand` | Expands terse slides with definitions and context from the document itself |
| 9 | `enhance` | Adds examples, analogies and revision points where density allows |
| 10 | `density` | Applies the text-density setting and paginates overflow into numbered continuations |
| 11 | `quiz` | Generates checkpoint quizzes and discussion prompts |
| 12 | `illustrate` | Chooses and draws illustrations and topic diagrams |
| 13 | `labs` | Attaches built-in or generated interactive labs |
| 14 | `score` | Computes PSF, fact retention and engagement scores |

| # | Stage | What it does |
|---|---|---|
| 15 | `build_pptx` | Writes the PowerPoint in an isolated subprocess |
| 16 | `build_html` | Writes the interactive web deck |

## 5.2 Stage 1: reading the source

Learnova has a dedicated reader for each input type, all producing the same Markdown IR with per-slide metadata.

**PowerPoint** (`parsers/ppt_parser.py`). Reads titles, bullets with their levels, tables, SmartArt text, charts (as data), equations, speaker notes and pictures. Colours of the source theme are kept for the colour-DNA step.

**Slide PDF** (`parsers/slide_pdf.py`). A lecture deck exported to PDF is already one page per slide, but it has lost its structure. The reader recovers it from geometry: the title is the largest line near the top, bullet nesting comes from the x position of the bullet glyph, wrapped lines are re-joined, ruled grids become real tables (`pdf_tables.py`), title pages and section dividers are recognised, and pictures are kept as figures. Text that exists only inside a picture is read by OCR and stored as `figure_text`: it is used for captions and grounding but never becomes bullet points.

**Report PDF** (`parsers/pdf_parser.py`, `pdf_structure.py`). Uses font sizes and weights to rebuild the heading tree, so a long report breaks at its own section boundaries.

**Word** (`parsers/markdown_converter.py`, `docx_to_markdown`). Read natively with python-docx; heading styles are normalised so every document chapter becomes a `##` section.

**Text and topic** (`parsers/text_inputs.py`, `ai/lesson_writer.py`). Typed notes are used as they are; a one-line topic is expanded into a short lesson outline.

**Image anchoring.** Extracted pictures are attached to the section that discusses them in three steps: an exact heading match, then word overlap with the section body, then position. This keeps a figure with its text even after the teacher edits or reorders sections.

**Colour DNA** (`analysis/source_dna.py`). The dominant background and saturated brand colours of the source are measured and turned into a palette with readable contrast, so the new deck still looks like the same course.

## 5.3 Stage 2: planning the slides

The planner (`pipeline/planner.py`) never shortens text. For each source section it decides how many slides should carry it:

- a short section (about 90 words or less) becomes one slide;
- a long section is split at natural boundaries (paragraphs, list items, topic shifts) into slides of about 45 to 90 words, preceded by an overview slide that maps the part.

When a slide has too many points it is paginated into numbered continuations. With `LEARNOVA_USE_CLASS=1`, **CLASS** (cognitive-load-aware slide segmentation) chooses the break points instead of splitting evenly. It is a dynamic program in the style of Knuth–Plass line breaking. Given points $b_1 \dots b_m$, a hard cap $P$ and a soft target $T$ per slide, it minimises

$$J = \sum_k \Big[ L(G_k) + \mu\,(1 - E(G_k)) + \nu\,\max(0, |G_k| - T)^2 \Big] + \lambda K$$

where $L$ is the cognitive load of a group, $E$ its information efficiency and $K$ the number of slides. It runs in $O(mP)$ time and places every point exactly once, so no content is dropped.

`slide_chunks.collapse_buildups` merges a teacher's build-up slides (the same slide repeated with one more bullet each time) into the complete one. Thin continuation slides are merged rather than enlarged.

# 6. The content decision engine

## 6.1 Role

`visuals/engine.py` is the heart of Learnova. It reads one slide and decides **how the idea should be shown**: as a process, a table, a flowchart, an algorithm box, an equation panel and so on. It returns the family, the data to draw it, a confidence and a plain-language reason that the preview shows to the teacher. It uses only rules, so it is fast, repeatable and free.

## 6.2 Visual families

| Family | Chosen when the slide contains |
|---|---|
| `PROCESS_LINEAR` | Three or more ordered steps or a described sequence |
| `PROCESS_CYCLIC` | Steps that return to the start (a cycle) |
| `FLOWCHART` | A procedure with a real decision (if/otherwise, until, check) and branches or loops |
| `ALGORITHM` | Pseudocode, numbered algorithm steps, code or markup; also a run of equations |
| `FORMULA_CALLOUT` | One key formula with its symbols |

| Family | Chosen when the slide contains |
|---|---|
| `WORKED_EXAMPLE` | A problem with its solution steps |
| `TIMELINE` | Three or more dated events |
| `COMPARE_TABLE` | An explicit comparison of two or more things on shared attributes |
| `COMPARE_VISUAL` | A two-sided comparison better shown side by side |
| `CAUSE_EFFECT` | Causes leading to effects |
| `STAT_TILES` | Two or more numeric facts |
| `CHART_TREND`, `CHART_CATEGORICAL`, `CHART_PART_TO_WHOLE` | Series, categories, or shares of a whole (including training logs) |
| `FUNNEL`, `LAYERS`, `HIERARCHY_NEST`, `MATRIX_GRID`, `SET_DIAGRAM` | Narrowing stages, stacked layers, nesting, 2×2 matrices, overlaps |
| `DEFINITION` | A term and its definition |
| `LIST_STRUCTURED` | Questions with answers (shown as flip cards) or labelled items |
| `CHECKLIST`, `KEY_POINTS`, `QUOTE`, `OUTLINE`, `SECTIONS`, `REFERENCES` | Requirements, main points, quotations, agenda, section dividers, citations |
| `FUNCTION_PLOT`, `GEOMETRY`, `CALCULUS_VIZ`, `ML_VIZ`, `DATA_STRUCTURE`, `ALGORITHM_TRACE` | Subject-specific drawings |
| `PROSE` | Running text that should stay as text (kept readable, not forced into a shape) |

## 6.3 Detector order

Detectors run in a fixed order so that a stronger, more specific signal wins over a general one:

1. tables in the source;
2. section dividers and reference lists;
3. **series** (a training log or numeric series becomes a trend chart), checked before table debris is filtered;
4. removal of pipe-character debris left by broken table exports;
5. **code** (`det_code`): OCR errors in code are repaired, statements are split without breaking strings or brackets, program output is separated into a console, and each line can get a short note;
6. **questions and answers** (`det_qa`): a wh-question with its answer becomes a flip card;
7. **algorithm** (`det_algorithm`): code, mathematical steps or markup, with indentation recovered

when the export lost it;

8. all remaining candidates, scored together, with the rule that a flowchart is chosen only when a real branch exists.

### 6.4 Rules for flowcharts, algorithms, code and equations

**Flowchart.** A decision node needs a real branch: words such as *otherwise*, *else*, *until*, a question mark or *check*. Questions that start with *what*, *why* or *how* are not decisions. The slide must also have sequence cues. Nodes are typed as start, process, decision and end; a decision records where *Yes* continues, where *No* goes and whether it loops back. The layout puts at most seven nodes in a row and routes loop-back arrows around the boxes so they never cross text.

**Algorithm.** All steps go into **one indented box** with line numbers, keywords in bold and nesting shown by indent. If the source lost its indentation (common in PDF exports), `_auto_indent` rebuilds it from block keywords such as *for*, *while*, *if* and *end*. Ordinary prose sentences are excluded so a paragraph is never mistaken for code.

**Code.** Code is shown in a dark editor-style panel with syntax colouring, a notes column that explains each statement in plain words, and a separate output console. In the web deck, hovering a note highlights its line.

**Equations.** `visuals/mathfmt.py` parses linear text such as `J(w,b) = (1/2m) Σ (h(x_i) − y_i)^2` into an expression tree and writes it three ways: as **Office Math (OMML)** so PowerPoint shows a real, editable equation exactly as Word does; as **MathML** in the web deck; and as clean Unicode as a fallback. LaTeX fragments in the source are converted first.

## 7. Enrichment

### 7.1 Key terms and glossary

`pipeline/enrich.py` collects terms the document itself defines ("X is …", "X refers to …", abbreviations in brackets). Any slide that uses a term defined elsewhere gets a *Key terms* strip with that definition, and a glossary slide closes the deck. Nothing is invented: every definition is quoted from the source.

### 7.2 Checkpoint quizzes

Quizzes are generated offline (`ai/quiz_offline.py`) from structured data already on the slides: step order, definitions, table cells, dates, figures and formulae. Distractors are taken from the same deck and have the same word type as the answer, so a question cannot be answered by grammar alone. A quiz appears every four content slides (more often for very long decks), with at most twenty per deck. An AI path exists but is opt-in.

## 7.3 Discussion prompts

Each slide gets a *think about it* question chosen from its visual and topic (a process asks which step matters most, a comparison asks when one would pick each side). It is shown on every third content slide, between quizzes, and placed in the speaker notes elsewhere.

## 7.4 Step-by-step explainers

`explain/detect.py` recognises worked problems in the text and `explain/solvers.py` solves them, writing every step with its reason. Supported: linear and quadratic equations, systems, percentages, simple and compound interest, statistics, probability, Bayes' rule, derivatives, integrals, geometry, and the NLP problems **CYK parsing, bigram probability, Naive Bayes and Viterbi**. Every number comes from SymPy or exact fractions. If no solver recognises a problem and a key is available, an AI-written explainer is produced, clearly labelled, at most six per deck.

## 7.5 Interactive labs

Labs are small simulations a student can play with. Built in: region growing, CYK parsing, Viterbi/HMM, bigram model, Naive Bayes word-sense disambiguation and HMM part-of-speech tagging. For other topics `labs/auto.py` makes one during the build: with a key, the model writes the lab logic, which is syntax-checked with Node before use; otherwise the slide's own steps, comparison rows or timeline events become a step-through. Labs open over the slide in the web deck and from a button in the PowerPoint.

## 7.6 Illustrations and topic diagrams

`ai/image_policy.py` decides *whether* a picture helps: concept slides yes; charts, tables, timelines and quizzes no; never two in a row. `ai/topic_art.py` classifies what the slide needs drawn (flow, plot, anatomy, comparison, scene or nothing). Offline, a scene is composed from theme-coloured Lucide icons. When a mechanism must be drawn and a key is available, the model writes an SVG with numbered, titled parts. The SVG is parsed and stripped of anything other than plain drawing elements (no script, links, external images or CSS), rendered with resvg, and cached. Its linked page lets a student step through the parts.

# 8. Rendering

## 8.1 PowerPoint builder

`rendering/ppt_builder.py` assembles the deck; `rendering/pptx_visuals.py` draws each family with native, editable shapes. Key points:

- **Layout fitting** (`rendering/layout.py`). Text is measured glyph by glyph and given the largest size that fits, stepping down in half points to a legibility floor (11 pt body, 9 pt cards). Below the floor the content paginates instead of shrinking. Sizes are also capped, so short text is never inflated to fill empty space.
- **Grids.** Cards in a row share one size; grids wrap past four per row and balance the last row (five become 3 + 2).
- **Wide families.** Algorithm boxes and flowcharts use the full slide width.
- **"In plain words" panel.** Explanation text the visual cannot carry is shown in the slide body at 11–14 pt. Families that already explain themselves (flowchart, algorithm, comparison table) do not get the panel.
- **Footer rows.** Key terms and, on every third slide, a *Discuss* prompt.
- **Speaker notes** carry the full source text, the discussion question and the reason for the chosen visual.
- **Equations** are inserted as OMML inside an `mc:AlternateContent` block, so PowerPoint shows a real equation and older viewers show the fallback picture.

## 8.2 Animations and transitions

python-pptx has no API for animation, so `rendering/pptx_animation.py` writes the XML directly at the schema-correct position (otherwise PowerPoint offers to repair the file). Each slide's parts appear on click in teaching order, merged into at most eight steps. `rendering/deck_director.py` chooses the transition between two slides from their relationship: continuation, next step on the same topic, sub-topic shift, new section or checkpoint.

## 8.3 Build isolation

PPTX and HTML builds run in a fresh interpreter (`rendering/subprocess_builder.py` with `build_worker.py`) started in its own process group. This removed crashes caused by C-extension state and network connection pools being copied into forked processes on macOS.

## 8.4 Interactive web deck

`rendering/web_deck_builder.py` writes one HTML file. `rendering/family_blocks.py` draws each family as HTML or SVG: the flowchart as inline SVG, the algorithm box with numbered lines, code with hover notes, equations in MathML. The deck includes click-by-click reveals, flip cards, a click-to-match activity, figure zoom, the original source slide on demand, quizzes with feedback, explainer overlays and labs.

**Phone mode.** Below 760 px width the deck switches off the fixed 16:9 layout: columns stack, each slide scrolls, and automatic font fitting is skipped, so text stays readable on a phone.

# 9. The AI layer

## 9.1 Offline-first escalation

`ai/escalation.py` decides, place by place, whether a model may be called. The rule-based result is always computed first.

| Place | The offline result is weak when … |
|---|---|
| Picture reading | Tesseract's text does not read as language (dictionary check in `textutils.is_readable`) |
| Titles and explanation | The title is a bare topic label, a counter or a generic word, or the slide is thin (under 60 words) |
| Topic diagram | The slide needs a mechanism drawn (flow, plot, anatomy, comparison) that offline scenes cannot show |
| Lab | The slide has too little structure for the offline step-through |
| Visual choice | The decision engine's confidence is below 0.7 |
| Explainer | No exact solver recognises the problem |
| Takeaway | The slide title is weak |

`LEARNOVA_AI_MODE` sets the policy: `auto` (default, escalate only when weak), `always`, or `off`. Every decision is counted, so each run logs how many places stayed offline. On eight test inputs, the system escalated for 37% of pictures, 30% of diagrams, 22% of takeaways and only 3% of titles; everything else stayed offline.

## 9.2 Provider router and key pools

`providers/router.py` exposes one interface for all model calls. For each task it has an ordered provider list and falls through on HTTP 429, time-outs and 5xx errors. A per-provider time-out floor stops a slow fallback model from always timing out. Multiple free keys can be given as comma-separated pools (`GROQ_API_KEYS`, `GEMINI_API_KEYS`). A key that hits its quota is set aside for fifteen minutes and the next one is used; Gemini calls are paced 4.2 seconds apart per key. Keys from one account share that account's limit, so pools help only when keys belong to different accounts. Identical requests are cached on disk (`LEARNOVA_LLM_CACHE`).

### 9.3 Checks on AI output

| Output | Check before use |
|---|---|
| SVG diagram | Must parse; only plain drawing elements allowed; rendered with resvg; otherwise offline scene |
| Lab JavaScript | Syntax-checked with Node; otherwise offline step-through |
| Rewritten text | Diffed against the source; unrepresented sentences appended back verbatim |
| Tutor answer | Built from retrieved passages only, cites slides as [S3], says when the deck does not cover a question |
| Explainer | Labelled as AI-written; exact solvers always preferred |

## 10. Quality measurement

### 10.1 Pedagogical Slide Fitness (PSF)

`scoring/psf.py` scores each slide from three parts in $[0, 1]$: information efficiency $E$ (new concepts, weighted by rarity, minus redundancy), cognitive load $L$ (concepts, words, visual elements and split attention through a sigmoid) and multimedia coherence $C$ (signalling, coherence, non-redundancy and spatial contiguity, from Mayer's principles). The score is

$$\mathrm{PSF}(s) = E(s)^{0.40}\,\big(1 - L(s)\big)^{0.42}\,C(s)^{0.18}$$

and the deck score is the geometric mean of slide scores multiplied by a flow term that rewards moderate overlap between consecutive slides.

### 10.2 Layout audit

`evaluation/layout_audit.py` inspects the shapes of a finished PPTX without rendering it and reports, per slide: coverage of the content band, empty space at the bottom, sparse cards, overflow past the slide edge, words broken across lines and the smallest font used. On nine unseen inputs (482 slides) the share of flagged slides fell from 4.1% to 2.3%.

### 10.3 Fact retention and before/after

`analysis/compare.py` compares each output slide with its source: the visual chosen and why, the change in word count and which figures, dates, formulae and terms were kept. It is shown in the app as *Before vs after*.

### 10.4 Automated tests

The pytest suite has 774 passing tests, covering parsers, every detector, the math formatter, flowchart and code detection, escalation rules, the router, rendering, the API routes and the storage layer. Gold sets in `tests/fixtures/` fix the expected visual for real slides, so a change that picks worse visuals fails. Tests run with all API keys blanked, so they never depend on the network.

## 11. Backend service

### 11.1 API endpoints

| Group | Endpoints | Purpose |
|---|---|---|
| Auth | `POST /api/auth/otp/request`, `/otp/verify`, `GET /api/auth/me`, `PUT /api/auth/profile`, `POST /logout`, `/logout-all`, `DELETE /api/auth/account` | Phone + OTP sign-in, profile, sessions, account deletion |
| Jobs | `POST /api/jobs`, `POST /api/jobs/typed`, `GET /api/jobs`, `GET/DELETE /api/jobs/{id}`, `POST /cancel` | Upload or type, poll, cancel |
| Review | `GET/PUT /api/jobs/{id}/markdown`, `POST /api/jobs/{id}/generate` | Edit the IR, start the build |
| Results | `GET /api/jobs/{id}/deck`, `/download/{pptx\|html}`, `/analysis` | Deck JSON, files, before/after |

| Group | Endpoints | Purpose |
|---|---|---|
| Library | `GET /api/decks`, `GET/DELETE /api/decks/{id}`, `/thumb`, `/markdown`, `/deck`, `/download/{artifact}` | Saved decks |
| Editing | `GET /editable`, `PUT /slides`, `POST /refresh`, `POST /versions/{v}/restore`, `GET/PUT /images/{slide}` | Edit slides, versions, replace images |
| Teaching | `POST /api/explain`, `GET /api/decks/{id}/explainers[/{slide}]`, `GET/POST /api/labs`, `GET /api/labs/{id}`, `GET /api/art/{id}` | Explainers, labs, diagram pages |
| Study | `GET /api/decks/{id}/export/{kind}`, `/narration`, `POST /api/decks/{id}/chat`, `GET /api/tutor/personas` | Study pack, narration, tutor |
| Sharing | `POST/GET /api/decks/{id}/shares`, `DELETE /shares/{token}`, `GET /api/share/{token}` | View-only links |
| Catalogue | `GET /api/templates`, `/themes`, `/palettes`, `/samples`, `/samples/{id}/file`, `/gallery…` | Templates, themes, sample files, gallery |
| AI keys | `GET/POST/DELETE /api/providers`, `GET /api/assist/status`, `POST /api/assist/slide`, `/prompt` | Manage keys, AI assist |
| Assistant | `POST /api/assistant/query`, `GET /registry`, `/intents`, `/session/{id}` | Chat and voice control |

| Group | Endpoints | Purpose |
|---|---|---|
| Health | `GET /api/health`, `GET /api/config`, `POST /api/events`, `/errors` | Status, client telemetry |

## 11.2 Data and storage

| Data | Where | Notes |
|---|---|---|
| Users, sessions, OTP hashes | SQLite `.data/learnova.db` | Only hashes of codes and session tokens are stored |
| Job records | Same SQLite file | Survive restarts; mid-build jobs marked interrupted |
| Saved decks | `.data/users/<user_id>/<deck_id>/` | Markdown, deck JSON, PPTX, HTML, metadata, versions |
| Share tokens | Deck folder | 128-bit tokens, can expire and be revoked |
| AI keys added in Settings | Server-side settings file | Never sent back to the browser |
| Diagram and LLM caches | `.cache/` | Keyed by content hash |
| Sample files | `data/samples/` with `samples.json` | Shown on the Templates page |

## 11.3 Security

- Sign-in uses a 6-digit code valid for five minutes, five attempts, single use, stored only as an HMAC.
- Session tokens are 256-bit random strings; only their SHA-256 is stored, so a leaked database cannot be replayed.
- The user id is always taken from the session, never from the request; a deck that belongs to someone else returns 404, not 403, so ids cannot be probed.
- Per-IP rate limits and security headers are on by default.
- Uploads are checked for type and size; page and chunk counts are capped (`LEARNOVA_MAX_PAGES`, `LEARNOVA_MAX_CHUNKS`).
- Generated SVG and JavaScript are sanitised or checked before they reach a page.
- `.env` and keys are never committed; the repository ships `.env.example` only.

# 12. Frontend

## 12.1 Structure

The React app in `frontend/src` has a public landing page and a signed-in workspace. Main pages:

| Page | Purpose |
|---|---|
| Landing, AuthPage, Onboarding | Introduction, phone sign-in, first-run setup |
| Dashboard (`/app`) | The composer: paste notes, upload a file or type a topic; progress; recent decks |
| Library / Templates | Ready-made lessons, sample lecture files (NLP and ML) to generate from, decks Learnova already made |
| Presentations, DeckLibrary, Projects | Saved decks and versions |
| Preview, Studio | Slide list, inspector with the chosen visual and reason, editing |
| Present, Audience | Presenter console with notes, timer, next slide, and a synced audience window |
| Compare | Before vs after |
| Explain | Solve a problem step by step |
| Quizzes, Analytics | Quiz runner, deck analytics |
| Export, Settings | Study pack, AI keys, theme |

`src/api.js` wraps every endpoint and attaches the session token. During development Vite proxies `/api` to port 8000, so no CORS setup is needed; in production the API serves the built app itself (`LEARNOVA_SERVE_WEB=1`).

## 12.2 Responsive design and PWA

The app has a bottom tab bar under 768 px, never scrolls sideways, and the landing page has its own phone layout. A manifest and service worker make it installable on Android, iOS and desktop, with shortcuts for *New deck*, *My decks* and *Solve step by step*.

# 13. Rules applied to every deck

These rules are enforced in code; most have tests. The full list in execution order is in `docs/PPT_RULES.md`.

## 13.1 Content rules

1. Every important point of the source is kept: on the slide, in the plain-words panel or in the speaker notes.
2. Nothing is invented. A table with no parseable rows or a metric with no real quantity falls back to text.
3. Text read from pictures never becomes bullet points; OCR captions appear only when readable.
4. Build-up slides are merged into the complete slide; thin continuations are merged, not enlarged.
5. Titles are in sentence case, without stray bullets or repeated words; generic titles ("Introduction", "Overview", "Slide 3") are replaced with the slide's own claim.
6. Duplicate sentences and restatements of the title are removed.
7. Quantities are kept whole (currency, commas, units); trimming always ends on a word boundary.

## 13.2 Visual rules

8. Each kind of content gets its own box: algorithms in one indented numbered box, code in an editor panel with explanations, equations as real equations, procedures with decisions as flowcharts.
9. A flowchart needs a real decision; a pros/cons visual needs both sides; a comparison needs shared attributes.
10. Illustrations only where they add meaning, never on charts, tables, timelines or quizzes, never two in a row.
11. The deck takes the colours of the source deck; text colour is chosen by WCAG contrast.

## 13.3 Layout rules

12. No overflow and no word broken across lines.
13. Font sizes fit the box but are capped; empty space is not filled by inflating type. Minimum 11 pt body, 9 pt cards.
14. A slide that would need smaller text is split into continuations instead (with CLASS when enabled).
15. Grids are balanced; algorithm boxes and flowcharts use the full width.

## 13.4 Interactivity rules

16. A checkpoint quiz every four content slides, at most twenty per deck, with same-type distractors.
17. A discussion prompt on every third content slide.
18. At most eight click-build steps per slide, in teaching order.
19. Worked problems become step-by-step explainers with computed numbers.

### 13.5 AI rules

20. Offline first: a model is called only when a specific weakness check fails.
21. Only free API keys are used; a spent key is rotated out; when every quota is used up, the deck still finishes.
22. Every AI output is checked (parsed, sanitised, syntax-checked or grounded) before it is used.

## 14. Configuration

All settings are environment variables, read from `.env`. None are required.

| Variable | Default | Effect |
|---|---|---|
| `GROQ_API_KEY`, `GROQ_API_KEYS` | none | One key or a pool of free Groq keys |
| `GEMINI_API_KEY`, `GEMINI_API_KEYS` | none | One key or a pool of free Gemini keys |
| `NVIDIA_API_KEY` | none | NVIDIA NIM failover |
| `GROQ_MODEL`, `GEMINI_MODEL`, `NVIDIA_MODEL` | see §3.2 | Override default models |
| `LEARNOVA_AI_MODE` | `auto` | `auto`, `always` or `off` |
| `LEARNOVA_LLM_EXPLAINERS` | 6 | Maximum AI-written explainers per deck |
| `LEARNOVA_AI_DIAGRAM_MAX` | 40 | Maximum AI topic diagrams per deck |
| `LEARNOVA_QUIZ_LLM` | off | Use a model for quizzes instead of the offline generator |
| `LEARNOVA_PPTX_ANIM` | 1 | `0` turns click-build animations off |
| `LEARNOVA_USE_CLASS` | off | `1` uses CLASS segmentation for overloaded slides |
| `LEARNOVA_LLM_CACHE` | on | Disk cache of identical model requests |
| `LEARNOVA_MAX_PAGES`, `LEARNOVA_MAX_CHUNKS`, `LEARNOVA_MAX_JOBS` | — | Input and concurrency limits |
| `LEARNOVA_RATE_LIMIT` | on | Per-IP rate limiting |

| Variable | Default | Effect |
|---|---|---|
| `LEARNOVA_DB_PATH` | `.data/learnova.db` | Database location |
| `LEARNOVA_SERVE_WEB` | off | Serve the built frontend from the API (production) |
| `LEARNOVA_PUBLIC_URL` | — | Base URL used in share links and diagram links |
| `OTP_DEV_MODE` | 1 | Show the sign-in code on screen (development only) |
| `OTP_SECRET`, `OTP_SMS_PROVIDER` | — | Production OTP hashing and SMS delivery |

# 15. Running and deployment

## 15.1 Local development

```
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn apps.api.main:app --port 8000
cd frontend && npm install && npm run dev -- --port 5190
```

Open `http://localhost:5190`; in development the sign-in code is shown on screen. A deck can also be built without the app: `python scripts/render_preview.py demo/corpus/bio_cycle.md`.

## 15.2 Docker

The `Dockerfile` has two stages: Node 20 builds the frontend, then a Python 3.12 image installs LibreOffice, Tesseract, fonts, Node and the Python requirements, copies the engine, the API, the sample data and the built frontend, and starts Uvicorn on port 8000.

```
docker build -t learnova .
docker run -p 8000:8000 --env-file .env learnova
```

## 15.3 Free hosting

| Host | Fit |
|---|---|
| Hugging Face Spaces (Docker) | Recommended: 2 vCPU and 16 GB RAM free, runs everything |

| Host | Fit |
|---|---|
| Render (Docker web service, `render.yaml`) | Works for small decks; 512 MB RAM, sleeps after 15 minutes |
| Vercel (`frontend/vercel.json`) | Web app only, forwarding `/api` to the API on Spaces or Render |
| Oracle Cloud Always Free VM | Runs everything permanently; self-managed |

Before inviting outside users: rotate all keys used during development and set `OTP_DEV_MODE=0` with a real SMS provider.

# 16. Known limitations and future work

- Lab notebooks exported to PDF where code sits beside a picture can still overflow on a few slides.
- Equations are verified in PowerPoint; LibreOffice previews show the fallback rendering.
- The job runner is single-process; scaling out needs a shared queue.
- Free model quotas are shared per account, so key pools help only across accounts.
- Planned: a classroom study to calibrate the PSF weights, Hindi and Marathi decks, voice narration, more NLP explainers (dependency parsing, TF-IDF, edit distance), nested-decision flowcharts and LMS export.